UE22EC252B-Machine Learning Algorithms

Expt-1

Aim: To access and observe the samples from

(a) iris dataset
(b) digits dataset
(c) diabetes dataset

1.a Program to access and observe the samples from iris dataset.

```
from sklearn import datasets
import pandas as pd



print('THIS IS  THE PROGRAM TO ACCESS  IRIS DATASET')
iris=datasets.load_iris()
print(" To print the description of Iris Dataset")
print(iris.DESCR)

print('\n\n\n\n')

# df will fold dataset as a table
df=pd.DataFrame(
   iris.data,
   columns=iris.feature_names
   )

#labels are assigned to df[target] table or array
df['target']=pd.Series(
    iris.target
   )

df['target_names']=df['target'].apply(lambda y:iris.target_names[y])

print('To display First 5 samples')

# df.head(5) will return the first five samples in the dataset
print(df.head(5))

print('To display randomply 5 samples')
#df.sample(5) will return randomly five samples from the dataset
print(df.sample(5))

# Train Test Split Ratio
from sklearn.model_selection import train_test_split
```

```
df_train,df_test=train_test_split(df,test_size=0.3) # For 70: 30 Split

print('The total number of samples in the dataset = ',df.shape[0])
print('The number of samples  in training set = ',df_train.shape[0])
print('The number of samples  in testing set = ',df_test.shape[0])


print('The first five samples of training set')
print(df_train.head(5))

print('\n\nThe first five samples of testing set')
print(df_test.head(5))
```

OUTPUT

```
THIS IS  THE PROGRAM TO ACCESS  IRIS DATASET
 To print the description of Iris Dataset
.. _iris_dataset:

Iris plants dataset
--------------------

**Data Set Characteristics:**

    :Number of Instances: 150 (50 in each of three classes)
    :Number of Attributes: 4 numeric, predictive attributes and the class
    :Attribute Information:
        - sepal length in cm
        - sepal width in cm
        - petal length in cm
        - petal width in cm
        - class:
                - Iris-Setosa
                - Iris-Versicolour
                - Iris-Virginica

    :Summary Statistics:

    ============== ==== ==== ======= ===== ====================
                    Min  Max   Mean    SD   Class Correlation
    ============== ==== ==== ======= ===== ====================
    sepal length:   4.3  7.9   5.84   0.83    0.7826
    sepal width:    2.0  4.4   3.05   0.43   -0.4194
    petal length:   1.0  6.9   3.76   1.76    0.9490  (high!)
    petal width:    0.1  2.5   1.20   0.76    0.9565  (high!)
    ============== ==== ==== ======= ===== ====================

    :Missing Attribute Values: None
    :Class Distribution: 33.3% for each of 3 classes.
    :Creator: R.A. Fisher
    :Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)
    :Date: July, 1988

The famous Iris database, first used by Sir R.A. Fisher. The dataset is taken
from Fisher's paper. Note that it's the same as in R, but not as in the UCI
```

Machine Learning Repository, which has two wrong data points.

This is perhaps the best known database to be found in the
pattern recognition literature. Fisher's paper is a classic in the field and
is referenced frequently to this day. (See Duda & Hart, for example.) The
data set contains 3 classes of 50 instances each, where each class refers to a
type of iris plant. One class is linearly separable from the other 2; the
latter are NOT linearly separable from each other.

.. topic:: References

- Fisher, R.A. "The use of multiple measurements in taxonomic problems" Annual Eugenics, 7, Part II, 179-188 (1936); also in "Contributions to Mathematical Statistics" (John Wiley, NY, 1950).
- Duda, R.O., & Hart, P.E. (1973) Pattern Classification and Scene Analysis. (Q327.D83) John Wiley & Sons. ISBN 0-471-22361-1. See page 218.
- Dasarathy, B.V. (1980) "Nosing Around the Neighborhood: A New System Structure and Classification Rule for Recognition in Partially Exposed Environments". IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. PAMI-2, No. 1, 67-71.
- Gates, G.W. (1972) "The Reduced Nearest Neighbor Rule". IEEE Transactions on Information Theory, May 1972, 431-433.
- See also: 1988 MLC Proceedings, 54-64. Cheeseman et al"s AUTOCLASS II conceptual clustering system finds 3 classes in the data.
- Many, many more ...

To display First 5 samples

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm) \
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

	target	target_names
0	0	setosa
1	0	setosa
2	0	setosa
3	0	setosa
4	0	setosa

To display randomply 5 samples

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm) \
3	4.6	3.1	1.5	0.2
94	5.6	2.7	4.2	1.3
55	5.7	2.8	4.5	1.3
107	7.3	2.9	6.3	1.8
139	6.9	3.1	5.4	2.1

	target	target_names
3	0	setosa
94	1	versicolor
55	1	versicolor
107	2	virginica

```
139      2    virginica
The total number of samples in the dataset =  150
The number of samples  in training set =  105
The number of samples  in testing set =  45
The first five samples of training set
    sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
104                6.5                3.0                5.8                2.2
23                 5.1                3.3                1.7                0.5
18                 5.7                3.8                1.7                0.3
52                 6.9                3.1                4.9                1.5
75                 6.6                3.0                4.4                1.4

    target target_names
104      2    virginica
23       0       setosa
18       0       setosa
52       1   versicolor
75       1   versicolor


The first five samples of testing set
    sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
55                 5.7                2.8                4.5                1.3
124                6.7                3.3                5.7                2.1
111                6.4                2.7                5.3                1.9
74                 6.4                2.9                4.3                1.3
76                 6.8                2.8                4.8                1.4

    target target_names
55       1   versicolor
124      2    virginica
111      2    virginica
74       1   versicolor
76       1   versicolor
```

Assignment

1) **Show that in two different executions same samples will not be present training and testing set**
2) **Give a reason, why target names appear as setosa, versicular and virginica?**
   Ans: The following command gives target names.

```
iris.target_names
```

OUTPUT: **array(['setosa', 'versicolor', 'virginica'], dtype='<U10')**

3) **Execute the following commands and give reason for the outputs obtained.**

```
print(df.shape)
print(df_train.shape)
print(df_test.shape)
```

1.b to access and observe the samples from iris dataset.

```
from sklearn import datasets
import pandas as pd



print('THIS IS  THE PROGRAM TO ACCESS  DIGITS DATASET')
digit=datasets.load_digits()
print(" To print the description of Digits Dataset")
print(digit.DESCR)

print('\n\n\n\n')

# df will fold dataset as a table
df=pd.DataFrame(
   digit.data,
   columns=digit.feature_names
   )

#labels are assigned to df[target] table or array
df['target']=pd.Series(
    digit.target
   )

df['target_names']=df['target'].apply(lambda y:digit.target_names[y])

print('To display First 5 samples')

# df.head(5) will return the first five samples in the dataset
print(df.head(5))

print('To display randomply 5 samples')
#df.sample(5) will return randomly five samples from the dataset
print(df.sample(5))

# Train Test Split Ratio
from sklearn.model_selection import train_test_split

df_train,df_test=train_test_split(df,test_size=0.3) # For 70: 30 Split

print('The total number of samples in the dataset = ',df.shape[0])
print('The number of samples  in training set = ',df_train.shape[0])
print('The number of samples  in testing set = ',df_test.shape[0])
```

```
print('The first five samples of training set')
print(df_train.head(5))

print('\n\nThe first five samples of testing set')
print(df_test.head(5))
```

OUTPUT

```
THIS IS  THE PROGRAM TO ACCESS  DIGITS DATASET

 To print the description of Digits Dataset

.. _digits_dataset:


Optical recognition of handwritten digits dataset

--------------------------------------------------


**Data Set Characteristics:**


    :Number of Instances: 1797

    :Number of Attributes: 64

    :Attribute Information: 8x8 image of integer pixels in the range
0..16.

    :Missing Attribute Values: None

    :Creator: E. Alpaydin (alpaydin '@' boun.edu.tr)

    :Date: July; 1998


This is a copy of the test set of the UCI ML hand-written digits
datasets

https://archive.ics.uci.edu/ml/datasets/Optical+Recognition+of+Handwrit
ten+Digits


The data set contains images of hand-written digits: 10 classes where

each class refers to a digit.


Preprocessing programs made available by NIST were used to extract

normalized bitmaps of handwritten digits from a preprinted form. From a

total of 43 people, 30 contributed to the training set and different 13
```

to the test set. 32x32 bitmaps are divided into nonoverlapping blocks of

4x4 and the number of on pixels are counted in each block. This generates

an input matrix of 8x8 where each element is an integer in the range

0..16. This reduces dimensionality and gives invariance to small

distortions.

For info on NIST preprocessing routines, see M. D. Garris, J. L. Blue, G.

T. Candela, D. L. Dimmick, J. Geist, P. J. Grother, S. A. Janet, and C.

L. Wilson, NIST Form-Based Handprint Recognition System, NISTIR 5469,

1994.

.. topic:: References

- C. Kaynak (1995) Methods of Combining Multiple Classifiers and Their

  Applications to Handwritten Digit Recognition, MSc Thesis, Institute of

  Graduate Studies in Science and Engineering, Bogazici University.

- E. Alpaydin, C. Kaynak (1998) Cascading Classifiers, Kybernetika.

- Ken Tang and Ponnuthurai N. Suganthan and Xi Yao and A. Kai Qin.

  Linear dimensionalityreduction using relevance weighted LDA. School of

  Electrical and Electronic Engineering Nanyang Technological University.

  2005.

- Claudio Gentile. A New Approximate Maximal Margin Classification

  Algorithm. NIPS. 2000.

To display First 5 samples

```
   pixel_0_0  pixel_0_1  pixel_0_2  pixel_0_3  pixel_0_4  pixel_0_5  \
0        0.0        0.0        5.0       13.0        9.0        1.0
1        0.0        0.0        0.0       12.0       13.0        5.0
2        0.0        0.0        0.0        4.0       15.0       12.0
3        0.0        0.0        7.0       15.0       13.0        1.0
4        0.0        0.0        0.0        1.0       11.0        0.0

   pixel_0_6  pixel_0_7  pixel_1_0  pixel_1_1  ...  pixel_7_0
pixel_7_1  \
0        0.0        0.0        0.0        0.0  ...        0.0
0.0
1        0.0        0.0        0.0        0.0  ...        0.0
0.0
2        0.0        0.0        0.0        0.0  ...        0.0
0.0
3        0.0        0.0        0.0        8.0  ...        0.0
0.0
4        0.0        0.0        0.0        0.0  ...        0.0
0.0

   pixel_7_2  pixel_7_3  pixel_7_4  pixel_7_5  pixel_7_6  pixel_7_7
target  \
0        6.0       13.0       10.0        0.0        0.0        0.0
0
1        0.0       11.0       16.0       10.0        0.0        0.0
1
2        0.0        3.0       11.0       16.0        9.0        0.0
2
3        7.0       13.0       13.0        9.0        0.0        0.0
3
4        0.0        2.0       16.0        4.0        0.0        0.0
4

   target_names
0             0
1             1
2             2
```

```
3              3
4              4


[5 rows x 66 columns]
To display randomply 5 samples
      pixel_0_0  pixel_0_1  pixel_0_2  pixel_0_3  pixel_0_4  pixel_0_5
\
527         0.0        0.0        6.0       13.0        0.0        0.0
95          0.0        0.0        0.0       11.0       16.0        8.0
162         0.0        5.0       16.0       16.0       16.0       11.0
1120        0.0        0.0        1.0       11.0       14.0        5.0
1295        0.0        0.0        4.0       15.0       13.0        3.0


      pixel_0_6  pixel_0_7  pixel_1_0  pixel_1_1  ...  pixel_7_0
pixel_7_1  \
527         0.0        0.0        0.0        0.0  ...        0.0
0.0
95          0.0        0.0        0.0        0.0  ...        0.0
0.0
162         1.0        0.0        0.0        4.0  ...        0.0
2.0
1120        0.0        0.0        0.0        0.0  ...        0.0
0.0
1295        0.0        0.0        0.0        4.0  ...        0.0
0.0


      pixel_7_2  pixel_7_3  pixel_7_4  pixel_7_5  pixel_7_6  pixel_7_7
\
527         6.0       16.0       16.0       16.0       16.0       12.0
95          0.0       12.0       16.0       15.0        0.0        0.0
162        15.0       16.0        9.0        0.0        0.0        0.0
1120        2.0       13.0       16.0        9.0        0.0        0.0
1295        5.0       15.0       16.0        5.0        0.0        0.0


      target  target_names
527        1             1
```

```
95         6             6
162        5             5
1120       1             1
1295       8             8

[5 rows x 66 columns]
The total number of samples in the dataset =  1797
The number of samples  in training set =  1257
The number of samples  in testing set =  540
The first five samples of training set
      pixel_0_0  pixel_0_1  pixel_0_2  pixel_0_3  pixel_0_4  pixel_0_5
\
4           0.0        0.0        0.0        1.0       11.0        0.0
53          0.0        0.0        4.0        8.0       16.0        5.0
1187        0.0        0.0        9.0       14.0       15.0        6.0
1686        0.0        0.0        8.0       14.0       12.0        3.0
1727        0.0        0.0        6.0       11.0       16.0       13.0

      pixel_0_6  pixel_0_7  pixel_1_0  pixel_1_1  ...  pixel_7_0
pixel_7_1  \
4           0.0        0.0        0.0        0.0  ...        0.0
0.0
53          0.0        0.0        0.0        0.0  ...        0.0
0.0
1187        0.0        0.0        0.0        2.0  ...        0.0
0.0
1686        0.0        0.0        0.0        6.0  ...        0.0
0.0
1727        5.0        0.0        0.0        2.0  ...        0.0
0.0

      pixel_7_2  pixel_7_3  pixel_7_4  pixel_7_5  pixel_7_6  pixel_7_7
\
4           0.0        2.0       16.0        4.0        0.0        0.0
53          6.0       16.0       12.0        1.0        0.0        0.0
1187        6.0       14.0        5.0        0.0        0.0        0.0
```

```
1686        7.0       16.0       16.0        8.0        0.0        0.0
1727        5.0       14.0       11.0        6.0        0.0        0.0

      target  target_names
4          4             4
53         8             8
1187       0             0
1686       9             9
1727       3             3

[5 rows x 66 columns]



The first five samples of testing set
     pixel_0_0  pixel_0_1  pixel_0_2  pixel_0_3  pixel_0_4  pixel_0_5
\
415        0.0        0.0        3.0       14.0       10.0        3.0
740        0.0        0.0        0.0        7.0       14.0       16.0
67         0.0        0.0        5.0       14.0        0.0        0.0
768        0.0        0.0        4.0       12.0       16.0        8.0
463        0.0        0.0       13.0       14.0        3.0        0.0

     pixel_0_6  pixel_0_7  pixel_1_0  pixel_1_1  ...  pixel_7_0
pixel_7_1  \
415        0.0        0.0        0.0        0.0  ...        0.0
0.0
740        6.0        0.0        0.0        0.0  ...        0.0
0.0
67         0.0        0.0        0.0        0.0  ...        0.0
0.0
768        0.0        0.0        0.0        5.0  ...        0.0
0.0
463        0.0        0.0        0.0        4.0  ...        0.0
0.0
```

```
     pixel_7_2  pixel_7_3  pixel_7_4  pixel_7_5  pixel_7_6  pixel_7_7
target  \
415        7.0       12.0       14.0       14.0        6.0        0.0
9
740        0.0        9.0        6.0        0.0        0.0        0.0
7
67         4.0       14.0       16.0       12.0        7.0        0.0
6
768        3.0       13.0       13.0       10.0        1.0        0.0
8
463       11.0       12.0       14.0       14.0        6.0        0.0
2


     target_names
415             9
740             7
67              6
768             8
463             2


[5 rows x 66 columns]
```

Assignment

1) **Show that in two different executions same samples will not be present training and testing set.**
2) **Give reason why target name is not appearing as One , two, three**

1.c to access and observe the samples from diabetis dataset.

```
from sklearn import datasets
import pandas as pd



print('THIS IS  THE PROGRAM TO ACCESS  DIABETES DATASET')
dbt=datasets.load_diabetes()
print(" To print the description of Diabetis Dataset")
print(dbt.DESCR)

print('\n\n\n\n')

# df will fold dataset as a table
```

```
df=pd.DataFrame(
   dbt.data,
   columns=dbt.feature_names
   )

#labels are assigned to df[target] table or array
df['target']=pd.Series(
    dbt.target
   )



print('To display First 5 samples')

# df.head(5) will return the first five samples in the dataset
print(df.head(5))

print('To display randomply 5 samples')
#df.sample(5) will return randomly five samples from the dataset
print(df.sample(5))

# Train Test Split Ratio
from sklearn.model_selection import train_test_split

df_train,df_test=train_test_split(df,test_size=0.3) # For 70: 30 Split

print('The total number of samples in the dataset = ',df.shape[0])
print('The number of samples  in training set = ',df_train.shape[0])
print('The number of samples  in testing set = ',df_test.shape[0])


print('The first five samples of training set')
print(df_train.head(5))

print('\n\nThe first five samples of testing set')
print(df_test.head(5))
```

OUTPUT

```
THIS IS  THE PROGRAM TO ACCESS  DIABETES DATASET

 To print the description of Digits Dataset

.. _diabetes_dataset:


Diabetes dataset

----------------
```

Ten baseline variables, age, sex, body mass index, average blood pressure, and six blood serum measurements were obtained for each of n = 442 diabetes patients, as well as the response of interest, a quantitative measure of disease progression one year after baseline.

Data Set Characteristics:

:Number of Instances: 442

:Number of Attributes: First 10 columns are numeric predictive values

:Target: Column 11 is a quantitative measure of disease progression one year after baseline

:Attribute Information:

- age age in years
- sex
- bmi body mass index
- bp average blood pressure
- s1 tc, total serum cholesterol
- s2 ldl, low-density lipoproteins
- s3 hdl, high-density lipoproteins
- s4 tch, total cholesterol / HDL
- s5 ltg, possibly log of serum triglycerides level
- s6 glu, blood sugar level

Note: Each of these 10 feature variables have been mean centered and scaled by the standard deviation times the square root of `n_samples` (i.e. the sum of squares of each column totals 1).

Source URL:

https://www4.stat.ncsu.edu/~boos/var.select/diabetes.html

For more information see:

Bradley Efron, Trevor Hastie, Iain Johnstone and Robert Tibshirani (2004) "Least Angle Regression," Annals of Statistics (with discussion), 407-499.

(https://web.stanford.edu/~hastie/Papers/LARS/LeastAngle_2002.pdf)

To display First 5 samples

```
        age       sex       bmi        bp        s1        s2        s3
\
0  0.038076  0.050680  0.061696  0.021872 -0.044223 -0.034821 -0.043401
1 -0.001882 -0.044642 -0.051474 -0.026328 -0.008449 -0.019163  0.074412
2  0.085299  0.050680  0.044451 -0.005670 -0.045599 -0.034194 -0.032356
3 -0.089063 -0.044642 -0.011595 -0.036656  0.012191  0.024991 -0.036038
4  0.005383 -0.044642 -0.036385  0.021872  0.003935  0.015596  0.008142

         s4        s5        s6  target
0 -0.002592  0.019907 -0.017646   151.0
1 -0.039493 -0.068332 -0.092204    75.0
2 -0.002592  0.002861 -0.025930   141.0
3  0.034309  0.022688 -0.009362   206.0
4 -0.002592 -0.031988 -0.046641   135.0
```

To display randomply 5 samples

```
          age       sex       bmi        bp        s1        s2
s3  \
261  0.048974 -0.044642 -0.041774  0.104501  0.035582 -0.025739
0.177497
263 -0.074533  0.050680 -0.077342 -0.046985 -0.046975 -0.032629
0.004460
337  0.019913  0.050680 -0.012673  0.070072 -0.011201  0.007141 -
0.039719
140  0.041708  0.050680  0.014272  0.042529 -0.030464 -0.001314 -
0.043401
```

```
363 -0.049105  0.050680 -0.024529  0.000079 -0.046975 -0.028245 -
0.065491


           s4        s5        s6  target

261 -0.076395 -0.012909  0.015491   103.0

263 -0.039493 -0.072133 -0.017646   116.0

337  0.034309  0.005386  0.003064    91.0

140 -0.002592 -0.033246  0.015491   118.0

363  0.028405  0.019196  0.011349    58.0

The total number of samples in the dataset =  442

The number of samples  in training set =  309

The number of samples  in testing set =  133

The first five samples of training set

          age       sex       bmi        bp        s1        s2
s3  \

168  0.001751  0.050680  0.059541 -0.002228  0.061725  0.063195 -
0.058127

295 -0.052738  0.050680  0.039062 -0.040099 -0.005697 -0.012900
0.011824

100  0.016281 -0.044642  0.017506 -0.022885  0.060349  0.044406
0.030232

269  0.009016 -0.044642 -0.032073 -0.026328  0.042462 -0.010395
0.159089

316  0.016281  0.050680  0.014272  0.001215  0.001183 -0.021355 -
0.032356


           s4        s5        s6  target

168  0.108111  0.068986  0.127328   268.0

295 -0.039493  0.016307  0.003064    85.0

100 -0.002592  0.037236 -0.001078   128.0

269 -0.076395 -0.011897 -0.038357    87.0

316  0.034309  0.074966  0.040343   220.0



The first five samples of testing set
```

```
          age       sex       bmi        bp        s1        s2
s3  \

435 -0.012780 -0.044642 -0.023451 -0.040099 -0.016704  0.004636 -
0.017629

271  0.038076  0.050680  0.008883  0.042529 -0.042848 -0.021042 -
0.039719

62  -0.027310  0.050680 -0.007284 -0.040099 -0.011201 -0.013840
0.059685

74   0.012648  0.050680  0.002417  0.056301  0.027326  0.017162
0.041277

413 -0.052738 -0.044642 -0.000817 -0.026328  0.010815  0.007141
0.048640


           s4        s5        s6  target

435 -0.002592 -0.038460 -0.038357    64.0

271 -0.002592 -0.018114  0.007207   127.0

62  -0.039493 -0.082379 -0.025930    52.0

74  -0.039493  0.003709  0.073480    85.0

413 -0.039493 -0.035816  0.019633   113.0
```

Assignment

1) **Show that in two different executions same samples will not be present training and testing set.**
2) **Give a reason, why the target name is not present?**

Expt 2: Aim: To split the dataset into train, validation, and test datasets.

(a) Iris dataset
(b) digits dataset
(c) diabetes dataset

2(a) Program to split the iris dataset into train, validation, and test datasets.

In this program, 30 % of the total samples are reserved for testing. Now 70% of data samples are divided in the ratio of 70:30 as training set and validation set.

```
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split

# load the iris dataset
iris = datasets.load_iris()
df = pd.DataFrame(iris.data)
label = pd.DataFrame(iris.target)



# Let us split 70% for Training and 30% for Testing

X_train, X_test, y_train, y_test = train_test_split(df,
label,test_size=0.3,shuffle = True)

X_train, X_val, y_train, y_val = train_test_split(X_train, y_train,
test_size=0.3,shuffle= True)


print(" Totoal number of samples in the Dataset  ", df.shape[0])
print(" Total number of labels in the Dataset ", label.shape[0])

print(" Total number of samples in training set X_train ",
X_train.shape[0])
print(" Total number of labels in training set y_train " ,
y_train.shape[0])

print(" Total number of samples in testing set X_test ",
X_test.shape[0])
print(" Total number of labels in testing set y_test " ,
y_test.shape[0])

print(" Total number of samples in validation set X_test ",
X_val.shape[0])
print(" Total number of labels in validation set  y_test " ,
y_val.shape[0])

print('The 3 samples of training set')
```

```
print(X_train.head(3))
print('The random 3 samples of training set')
print(X_train.sample(3))

print('The 3 samples of testing set')
print(X_test.head(3))
print('The random 3 samples of testing set')
print(X_test.sample(3))

print('The 3 samples of validation set')
print(X_val.head(3))
print('The random 3 samples of validation set')
print(X_val.sample(3))
```

OUTPUT

```
Total number of samples in the Dataset   150
 Total number of labels in the Dataset  150
 Total number of samples in training set X_train  73
 Total number of labels in training set y_train  73
 Total number of samples in testing set X_test  45
 Total number of labels in testing set y_test  45
 Total number of samples in validation set X_test  32
 Total number of labels in validation set  y_test  32
The 3 samples of training set
     0   1   2   3
100  6.3  3.3  6.0  2.5
65   6.7  3.1  4.4  1.4
86   6.7  3.1  4.7  1.5
The random 3 samples of training set
    0   1   2   3
29  4.7  3.2  1.6  0.2
63  6.1  2.9  4.7  1.4
67  5.8  2.7  4.1  1.0
The 3 samples of testing set
    0   1   2   3
61  5.9  3.0  4.2  1.5
53  5.5  2.3  4.0  1.3
55  5.7  2.8  4.5  1.3
The random 3 samples of testing set
    0   1   2   3
53  5.5  2.3  4.0  1.3
42  4.4  3.2  1.3  0.2
55  5.7  2.8  4.5  1.3
The 3 samples of validation set
     0   1   2   3
82   5.8  2.7  3.9  1.2
20   5.4  3.4  1.7  0.2
120  6.9  3.2  5.7  2.3
The random 3 samples of validation set
     0   1   2   3
108  6.7  2.5  5.8  1.8
20   5.4  3.4  1.7  0.2
81   5.5  2.4  3.7  1.0
```

Assignment

1) Obtain the results for train = 65% validation = 15% test =20% and justify.
2) Obtain the results for train = 70% validation = 15% test =15% and justify.
3) Obtain the results for train = 85% validation = 5% test =10% and justify.

2(b) Program to split the digits dataset into train, validation, and test datasets.

In this program, 20 % of the total samples are reserved for testing. Now 80% of data samples are divided in the ratio of 80:20 as training set and validation set.

```
#Program 2b
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split

# load the digits dataset
digit = datasets.load_digits()
df = pd.DataFrame(digit.data)
label = pd.DataFrame(digit.target)



#  Let us split 80% for Training and 20% for Testing
X_train, X_test, y_train, y_test = train_test_split(df, label,test_size=0.2, shuffle = True)



# Of the 80% Traing Set , Now 20% is reserved for validation set and remaining 80% is reserverd for
Training
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.2, shuffle = True)

print(" Total number of samples in the Dataset  ", df.shape[0])
print(" Total number of labels in the Dataset ", label.shape[0])

print(" Total number of samples in training set X_train ", X_train.shape[0])
print(" Total number of labels in training set y_train " , y_train.shape[0])

print(" Total number of samples in testing set X_test ", X_test.shape[0])
print(" Total number of labels in testing set y_test " , y_test.shape[0])

print(" Total number of samples in validation set X_test ", X_val.shape[0])
print(" Total number of labels in validation set  y_test " , y_val.shape[0])

print('The 3 samples of training set')
print(X_train.head(3))
print('The 3 random  samples of training set')
print(X_train.sample(3))
```

```
print('The 3 samples of validation set')
print(X_val.head(3))
print('The 3 random  samples of validation set')
print(X_val.sample(3))

print('The 3 samples of testing set')
print(X_test.head(3))
print('The 3 random  samples of testing set')
print(X_test.sample(3))
```

OUTPUT

```
Total number of samples in the Dataset   1797
 Total number of labels in the Dataset  1797
 Total number of samples in training set X_train  1149
 Total number of labels in training set y_train  1149
 Total number of samples in testing set X_test  360
 Total number of labels in testing set y_test  360
 Total number of samples in validation set X_test  288
 Total number of labels in validation set  y_test  288
The 3 samples of training set
     0   1    2   3    4    5    6   7   8   9  ...   54   55  \
146   0.0  0.0   0.0  6.0  14.0   1.0   0.0  0.0  0.0  0.0  ...  12.0  0.0
1124  0.0  0.0   0.0  8.0  12.0   0.0   0.0  0.0  0.0  0.0  ...   0.0  0.0
1113  0.0  0.0  11.0  8.0   4.0  13.0  16.0  3.0  0.0  2.0  ...   0.0  0.0

     56   57   58   59   60   61   62   63
146   0.0  0.0   0.0  9.0  15.0  14.0  6.0  0.0
1124  0.0  0.0   0.0  12.0  10.0   0.0  0.0  0.0
1113  0.0  0.0  16.0   7.0   0.0   0.0  0.0  0.0

[3 rows x 64 columns]
The 3 random  samples of training set
     0   1   2   3    4    5   6   7   8   9  ...   54   55   56  \
982   0.0  0.0  8.0  11.0   0.0   0.0  0.0  0.0  0.0  0.0  ...  8.0  5.0  0.0
1080  0.0  0.0  9.0  16.0   6.0   0.0  0.0  0.0  0.0  3.0  ...  0.0  0.0  0.0
703   0.0  0.0  1.0  13.0  16.0  10.0  0.0  0.0  0.0  1.0  ...  9.0  0.0  0.0

     57   58   59   60   61   62   63
982   0.0   4.0  15.0  16.0  16.0  16.0  16.0
1080  0.0  12.0  14.0   3.0   0.0   0.0   0.0
703   0.0   0.0  14.0  16.0  11.0   0.0   0.0

[3 rows x 64 columns]
The 3 samples of validation set
     0   1    2    3    4    5   6   7   8    9  ...   54   55  \
1031  0.0  1.0  15.0  16.0   4.0   0.0  0.0  0.0  0.0   9.0  ...  12.0  1.0
807   0.0  0.0   2.0  10.0  10.0  11.0  0.0  0.0  0.0   0.0  ...   1.0  0.0
1589  0.0  4.0  16.0  16.0  16.0  15.0  3.0  0.0  0.0  11.0  ...   0.0  0.0

     56   57   58   59   60   61   62   63
1031  0.0  0.0  13.0  16.0  16.0  15.0  11.0  1.0
807   0.0  0.0   2.0  14.0  16.0  13.0   0.0  0.0
```

```
1589  0.0  5.0  16.0   3.0   0.0   0.0   0.0  0.0

[3 rows x 64 columns]
The 3 random  samples of validation set
     0   1   2    3    4    5   6   7   8   9  ...  54  55  56 \
1328  0.0  0.0  0.0   2.0  15.0   8.0  0.0  0.0  0.0  0.0  ...  0.0  0.0  0.0
438   0.0  0.0  2.0  12.0  12.0  12.0  9.0  2.0  0.0  0.0  ...  0.0  0.0  0.0
1082  0.0  0.0  6.0  16.0  15.0   5.0  0.0  0.0  0.0  1.0  ...  8.0  0.0  0.0

     57   58   59   60   61   62   63
1328  0.0  0.0   2.0  15.0   5.0  0.0  0.0
438   0.0  3.0  15.0   3.0   0.0  0.0  0.0
1082  0.0  8.0  16.0  16.0  12.0  1.0  0.0

[3 rows x 64 columns]
The 3 samples of testing set
    0   1    2    3    4    5   6   7   8   9  ...  54  55  56 \
154  0.0  0.0   0.0   3.0  16.0   3.0  0.0  0.0  0.0  0.0  ...  0.0  0.0  0.0
867  0.0  0.0  10.0  16.0  16.0   4.0  0.0  0.0  0.0  0.0  ...  2.0  0.0  0.0
862  0.0  0.0   5.0  16.0  16.0  16.0  9.0  0.0  0.0  0.0  ...  0.0  0.0  0.0

    57   58   59   60   61   62   63
154  0.0   0.0   4.0  12.0   0.0  0.0  0.0
867  0.0  11.0  16.0  16.0  10.0  0.0  0.0
862  0.0  11.0  10.0   0.0   0.0  0.0  0.0

[3 rows x 64 columns]
The 3 random  samples of testing set
     0   1   2    3    4   5   6   7   8   9  ...   54  55  56 \
398   0.0  0.0  7.0  13.0   3.0  0.0  0.0  0.0  0.0  0.0  ...  16.0  3.0  0.0
287   0.0  0.0  3.0  11.0  13.0  5.0  0.0  0.0  0.0  0.0  ...  11.0  0.0  0.0
1115  0.0  0.0  1.0  13.0   2.0  0.0  0.0  0.0  0.0  0.0  ...  14.0  5.0  0.0

     57   58   59   60   61   62   63
398   0.0  6.0  15.0   6.0   9.0   9.0  1.0
287   0.0  3.0   9.0  16.0  16.0   6.0  0.0
1115  0.0  1.0  11.0  16.0  16.0  13.0  1.0

[3 rows x 64 columns]
```

Assignment

1) **In the above program replace shuffle = True by shuffle = False. Execute two times, note the result and justify the result.**

2) Split the diabetes dataset in training , testing and validation sets. 25% of the total dataset is reserved for testing set. Out of 75% of the remaining samples, training and validation sets are divided in the ratio 70:30

Expt:-3

Aim:-To perform K-Fold cross-validation. Write a program to Split the given dataset into K-train and test sets using K-Fold cross-validation.

```
from sklearn.datasets import make_classification
import pandas as pd


X,y=make_classification(n_samples=10,n_features=4,n_classes=2)
import numpy as np
def kfold_indices(data, k):
    fold_size = len(data) // k
    indices = np.arange(len(data))
    folds = []
    for i in range(k):
        test_indices = indices[i * fold_size: (i + 1) * fold_size]
        train_indices = np.concatenate([indices[:i * fold_size],
indices[(i + 1) * fold_size:]])
        folds.append((train_indices, test_indices))
    return folds

# Define the number of folds (K)
k = 5

# Get the fold indices
fold_indices = kfold_indices(X, k)


from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier  # import ML  model

# Initialize the machine learning model
model = DecisionTreeClassifier()

# Initialize a list to store the evaluation scores
scores = []

# Iterate through each fold
for train_indices, test_indices in fold_indices:
    X_train, y_train = X[train_indices], y[train_indices]
    X_test, y_test = X[test_indices], y[test_indices]

    print('X_train samples')
    print(X_train)
    print('X_test samples')
    print(X_test)
    # Train the model on the training data
```

```
    model.fit(X_train, y_train)

    # Make predictions on the test data
    y_pred = model.predict(X_test)

    # Calculate the accuracy score for this fold
    fold_score = accuracy_score(y_test, y_pred)

    # Append the fold score to the list of scores
    scores.append(fold_score)

# Calculate the mean accuracy across all folds
mean_accuracy = np.mean(scores)
print("K-Fold Cross-Validation Scores:", scores)
print("Mean Accuracy:", mean_accuracy)
```

OUTPUT

```
X_train samples
[[-1.18941118  0.48413876  1.99104662  1.22821249]
 [ 1.56378906 -0.02472443 -0.58163875  1.03924999]
 [ 0.60791888 -0.46216114 -1.73221979 -1.5591981 ]
 [-1.6902142   0.14973539  1.03805245 -0.58963015]
 [-2.54374997  0.31212816  1.85105828 -0.51093257]
 [-0.76359532  0.28232836  1.18343782  0.66493047]
 [ 1.40146806  0.10232489 -0.10697891  1.471395  ]
 [-0.13135007 -0.15316023 -0.46778227 -0.76072441]]
X_test samples
[[-0.53713281 -0.18806132 -0.45435904 -1.20963223]
 [ 0.90410033 -0.44415591 -1.76687511 -1.26394141]]
X_train samples
[[-0.53713281 -0.18806132 -0.45435904 -1.20963223]
 [ 0.90410033 -0.44415591 -1.76687511 -1.26394141]
 [ 0.60791888 -0.46216114 -1.73221979 -1.5591981 ]
 [-1.6902142   0.14973539  1.03805245 -0.58963015]
 [-2.54374997  0.31212816  1.85105828 -0.51093257]
 [-0.76359532  0.28232836  1.18343782  0.66493047]
 [ 1.40146806  0.10232489 -0.10697891  1.471395  ]
 [-0.13135007 -0.15316023 -0.46778227 -0.76072441]]
X_test samples
[[-1.18941118  0.48413876  1.99104662  1.22821249]
 [ 1.56378906 -0.02472443 -0.58163875  1.03924999]]
X_train samples
[[-0.53713281 -0.18806132 -0.45435904 -1.20963223]
 [ 0.90410033 -0.44415591 -1.76687511 -1.26394141]
 [-1.18941118  0.48413876  1.99104662  1.22821249]
 [ 1.56378906 -0.02472443 -0.58163875  1.03924999]
 [-2.54374997  0.31212816  1.85105828 -0.51093257]
 [-0.76359532  0.28232836  1.18343782  0.66493047]
 [ 1.40146806  0.10232489 -0.10697891  1.471395  ]
 [-0.13135007 -0.15316023 -0.46778227 -0.76072441]]
X_test samples
[[ 0.60791888 -0.46216114 -1.73221979 -1.5591981 ]
 [-1.6902142   0.14973539  1.03805245 -0.58963015]]
```

```
X_train samples
[[-0.53713281 -0.18806132 -0.45435904 -1.20963223]
 [ 0.90410033 -0.44415591 -1.76687511 -1.26394141]
 [-1.18941118  0.48413876  1.99104662  1.22821249]
 [ 1.56378906 -0.02472443 -0.58163875  1.03924999]
 [ 0.60791888 -0.46216114 -1.73221979 -1.5591981 ]
 [-1.6902142   0.14973539  1.03805245 -0.58963015]
 [ 1.40146806  0.10232489 -0.10697891  1.471395  ]
 [-0.13135007 -0.15316023 -0.46778227 -0.76072441]]
X_test samples
[[-2.54374997  0.31212816  1.85105828 -0.51093257]
 [-0.76359532  0.28232836  1.18343782  0.66493047]]
X_train samples
[[-0.53713281 -0.18806132 -0.45435904 -1.20963223]
 [ 0.90410033 -0.44415591 -1.76687511 -1.26394141]
 [-1.18941118  0.48413876  1.99104662  1.22821249]
 [ 1.56378906 -0.02472443 -0.58163875  1.03924999]
 [ 0.60791888 -0.46216114 -1.73221979 -1.5591981 ]
 [-1.6902142   0.14973539  1.03805245 -0.58963015]
 [-2.54374997  0.31212816  1.85105828 -0.51093257]
 [-0.76359532  0.28232836  1.18343782  0.66493047]]
X_test samples
[[ 1.40146806  0.10232489 -0.10697891  1.471395  ]
 [-0.13135007 -0.15316023 -0.46778227 -0.76072441]]
K-Fold Cross-Validation Scores: [0.5, 1.0, 0.5, 1.0, 1.0]
Mean Accuracy: 0.8
```

Assignment

1) **Perform K-Fold Cross-validation with number of samples = 20 and number of splits = 5. Calculate mean accuracy. Justify K-Fold Cross-validation.**
2) **Perform K-Fold Cross validation with number of samples = 20 and number of splits = 10. Calculate mean accuracy. Justify K-Fold Cross-validation.**
3) **Compare the mean accuracy of the above two cases and give your inference.**

Expt-4

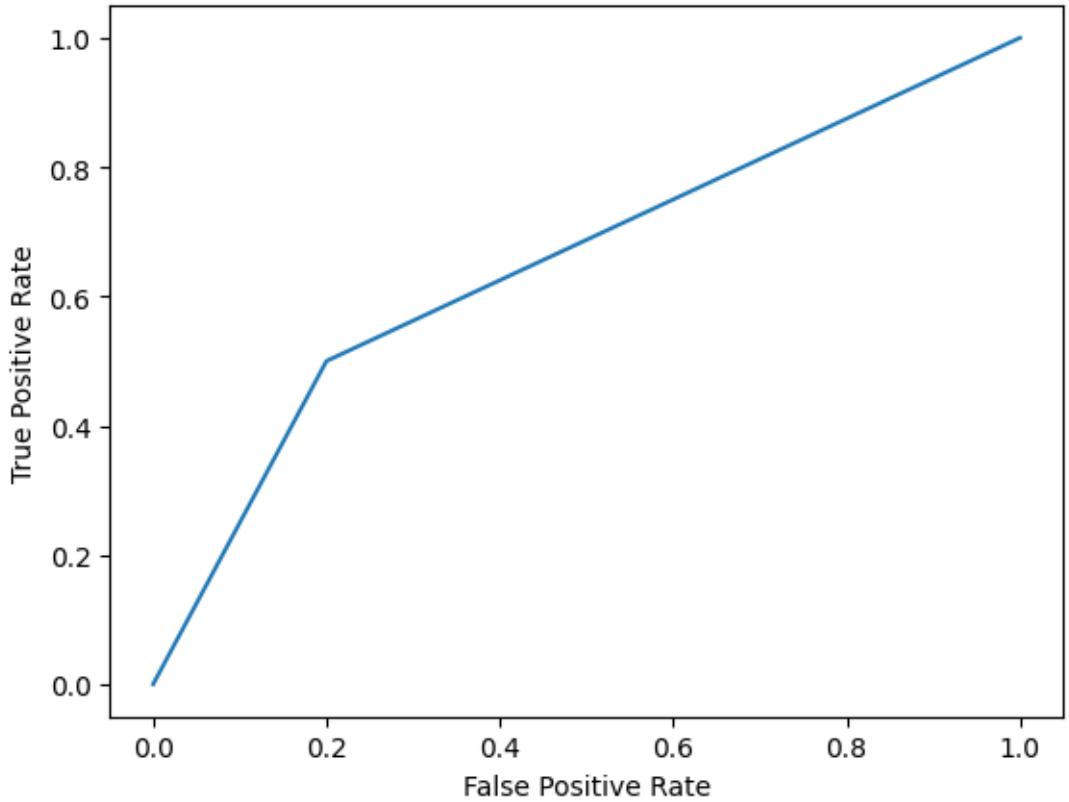
Aim: Generate a Confusion Matrix and compute true positive, true negative, false positive, and false negative.

Program to generate confusion Matrix and classification report

```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn import metrics
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score
# actual values
#A=1= Positive Class , B=0=Negative Class
actual = [1,0,0,1,0,1,1,1,0,1,0]
# predicted values
predicted = [1,0,0,1,0,0,0,1,0,0,1]
# confusion matrix
matrix = confusion_matrix(actual,predicted, labels=[1,0])
print('Confusion matrix : \n',matrix)
acc=accuracy_score(actual,predicted)
print('Accuracy = ',acc)
matrix = classification_report(actual,predicted,labels=[1,0])
print('Classification Report \n')
print(matrix)
fpr, tpr , _= metrics.roc_curve(actual, predicted) #create ROC curve
print('fpr = ',fpr)
print('tpr = ',tpr)
plt.plot(fpr,tpr)
plt.ylabel('True Positive Rate')
plt.xlabel('False Positive Rate')
plt.show()
```

OUTPUT

```
Confusion matrix :
 [[3 3]
 [1 4]]
Accuracy =  0.6363636363636364
Classification Report

              precision    recall  f1-score   support

           1       0.75      0.50      0.60         6
           0       0.57      0.80      0.67         5

    accuracy                           0.64        11
   macro avg       0.66      0.65      0.63        11
weighted avg       0.67      0.64      0.63        11

fpr =  [0.  0.2 1. ]
tpr =  [0.  0.5 1. ]
```



Assignment:

1) **Verify theoretically the entries of the classification report.**

   Note: $f1 - score = \frac{2*Precision*Recall}{Precision+Recall}$

2) **Experiment with the following actual and predicted samples and verify the entries of the classification report.**

```
# actual values
#A=1=Positive Class , B=0=Negative Class
actual = [1,0,0,1,0,1,1,1,0,1,0,1,1,1,1,0,0,1]
# predicted values
predicted = [1,0,0,1,0,0,0,1,0,0,1,0,0,0,0,0,1,0]
```

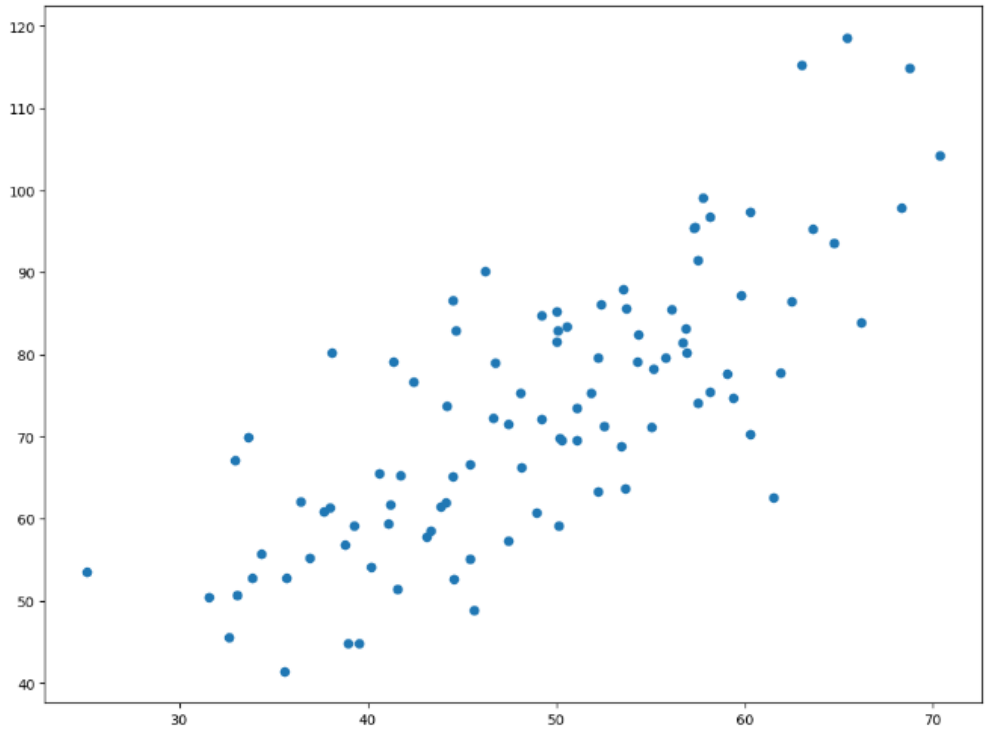
Program - 5

Program to learn Linear Regression using Gradient Descent method

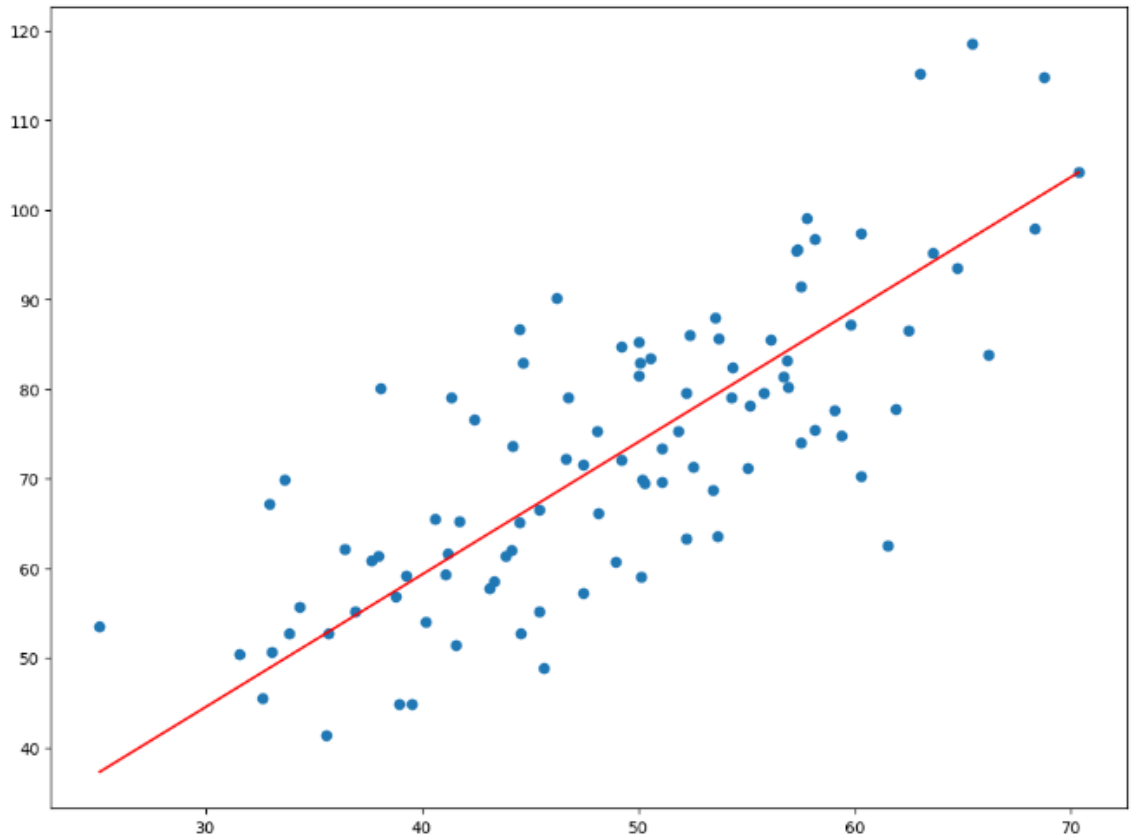
```
# Linear Regression with Gradient Descent
# Making the imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = (12.0, 9.0)
# Preprocessing Input data
data = pd.read_csv('data.csv')
X = data.iloc[:, 0]
Y = data.iloc[:, 1]
plt.scatter(X, Y)
plt.show()
# Building the model
m = 0
c = 0

L = 0.0001  # The learning Rate
epochs = 1000  # The number of iterations to perform gradient descent

n = float(len(X)) # Number of elements in X
```

```
# Performing Gradient Descent
for i in range(epochs):
    Y_pred = m*X + c  # The current predicted value of Y
    D_m = (-2/n) * sum(X * (Y - Y_pred))  # Derivative wrt m
    D_c = (-2/n) * sum(Y - Y_pred)  # Derivative wrt c
    m = m - L * D_m  # Update m
    c = c - L * D_c  # Update c
print (m, c)
# Making predictions
Y_pred = m*X + c
plt.scatter(X, Y)
plt.plot([min(X), max(X)], [min(Y_pred), max(Y_pred)],
color='red')  # regression line
plt.show()
```

OUTPUT




```
1.4796491688889395 0.10148121494753734
```



Assignment:

Write the inference for the linear regression model by varying 'm' and 'c' values (with reference to the regression line obtained)

Theory behind Linear Regression using Gradient Descent method:

Linear Regression

In statistics, linear regression is a linear approach to modelling the relationship between a dependent variable and one or more independent variables. Let **X** be the independent variable and **Y** be the dependent variable. Let us define a linear relationship between these two variables as follows:

$$Y = mX + c$$

where **m** is the slope of the line and **c** is the y intercept.

This equation is used to train our model with a given dataset and predict the value of **Y** for any given value of **X**. The challenge is to determine the value of **m** and **c**, such that the line corresponding to those values is the best fitting line or gives the minimum error.

Loss Function

The loss is the error in predicted value of **m** and **c**. Goal is to minimize this error to obtain the most accurate value of **m** and **c**. Loss is calculated using the Mean Squared Error function.

There are three steps in this function:

1. Find the difference between the actual Y and predicted Y value $(Y = mX + c)$, for a given X.
2. Square this difference.
3. Find the mean of the squares for every value in X.

The Gradient Descent Algorithm

Gradient descent is an iterative optimization algorithm to find the minimum of a function. In this program, GD function is used as Loss Function.

To understand the concept of Gradient Descent, imagine a valley and a person with no sense of direction who wants to get to the bottom of the valley. He goes down the slope and takes large steps when the slope is steep and small steps when the slope is less steep. He decides his next position based on his current position and stops when he gets to the bottom of the valley which was his goal.

Let's try applying gradient descent to **m** and **c** and approach it step by step:

1. Initially let m = 0 and c = 0. Let L be the learning rate. This controls how much the value of **m** changes with each step. L could be a small value like 0.0001 for good accuracy.
2. Calculate the partial derivative of the loss function with respect to m, and plug in the current values of x, y, m and c in it to obtain the derivative value **D**.

$$D_m = \frac{1}{n}\sum_{i=0}^{n} 2(y_i - (mx_i + c))(-x_i)$$

$$D_m = \frac{-2}{n}\sum_{i=0}^{n} x_i(y_i - \bar{y}_i)$$

D_m is the value of the partial derivative with respect to **m**. Similarly, find the partial derivative with respect to **c**, D_c :

$$D_c = \frac{-2}{n}\sum_{i=0}^{n}(y_i - \bar{y}_i)$$

3. Now we update the current value of **m** and **c** using the following equation:

$$m = m - L \times D_m$$

$$c = c - L \times D_c$$

4. Repeat this process until the loss function is a very small value or ideally 0 (which means 0 error or 100% accuracy). The value of **m** and **c** that are left with are the optimum values.

   With these, **m** can be considered the current position of the person. **D** is equivalent to the steepness of the slope and **L** can be the speed with which he moves. Now the new value of **m** that is used to calculate using the above equation will be his next position, and **L×D** will be the size of the steps he will take. When the slope is steeper (**D** is more) he takes longer steps and when it is less steep (**D** is less), he takes smaller steps. Finally, he arrives at the bottom of the valley which corresponds to loss = 0. Now with the optimum value of **m** and **c**, model is ready to make predictions.

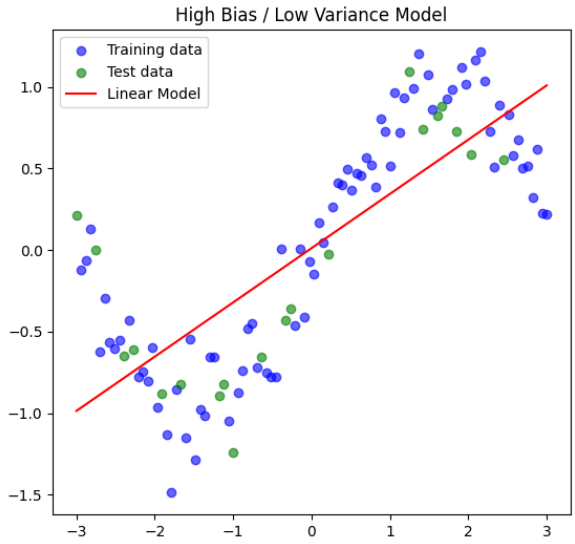
Reference:

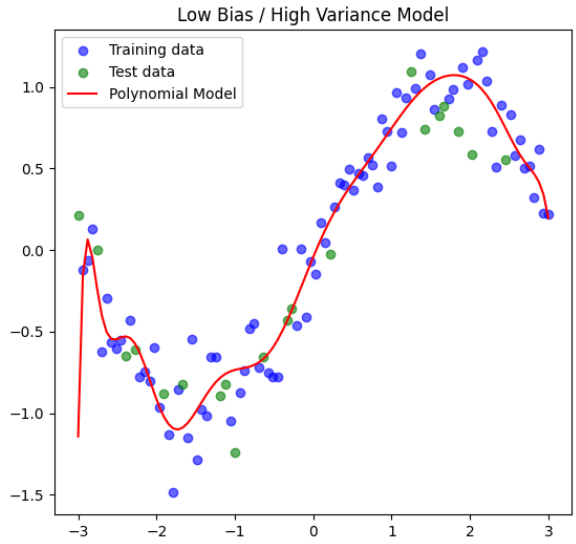
1. Linear Regression using Gradient Descent | by Adarsh Menon | Towards Data Science

```
#Expt6
#Aim:To Learn bias-variance trade-off
# estimate the bias and variance for a regression model
# Exhibiting low bias, high variance model and high bias, low variance
model
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
# Generate a synthetic dataset
np.random.seed(0)
X = np.linspace(-3, 3, 100)
y = np.sin(X) + np.random.normal(0, 0.2, 100) # Adding noise
X = X[:, np.newaxis] # Reshape for sklearn
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
# High Bias / Low Variance Model (Linear Regression)
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
y_pred_train_linear = linear_model.predict(X_train)
y_pred_test_linear = linear_model.predict(X_test)
# Low Bias / High Variance Model (Polynomial Regression with high
degree)
poly_model = make_pipeline(PolynomialFeatures(degree=15),
LinearRegression())
poly_model.fit(X_train, y_train)
y_pred_train_poly = poly_model.predict(X_train)
y_pred_test_poly = poly_model.predict(X_test)
# Plotting
plt.figure(figsize=(14, 6))
# Linear Model Plot
plt.subplot(1, 2, 1)
plt.scatter(X_train, y_train, label='Training data', color='blue',
alpha=0.6)
plt.scatter(X_test, y_test, label='Test data', color='green',
alpha=0.6)
plt.plot(X, linear_model.predict(X), color='red', label='Linear Model')
plt.title('High Bias / Low Variance Model')
plt.legend()
# Polynomial Model Plot
plt.subplot(1, 2, 2)
plt.scatter(X_train, y_train, label='Training data', color='blue',
alpha=0.6)
```

```
plt.scatter(X_test, y_test, label='Test data', color='green',
alpha=0.6)
plt.plot(np.linspace(-3, 3, 100), poly_model.predict(np.linspace(-3, 3,
100)[:, np.newaxis]), color='red', label='Polynomial Model')
plt.title('Low Bias / High Variance Model')
plt.legend()
plt.show()
# Print Mean Squared Error for both models
print(f"Linear Model Training MSE: {mean_squared_error(y_train,
y_pred_train_linear)}")
print(f"Linear Model Testing MSE: {mean_squared_error(y_test,
y_pred_test_linear)}")
print(f"Polynomial Model Training MSE: {mean_squared_error(y_train,
y_pred_train_poly)}")
print(f"Polynomial Model Testing MSE: {mean_squared_error(y_test,
y_pred_test_poly)}")
# Calculate Mean Squared Error (MSE) for training and testing sets
mse_train_linear = mean_squared_error(y_train, y_pred_train_linear)
mse_test_linear = mean_squared_error(y_test, y_pred_test_linear)
mse_train_poly = mean_squared_error(y_train, y_pred_train_poly)
mse_test_poly = mean_squared_error(y_test, y_pred_test_poly)
# Display MSE for both models
print("Linear Regression Model (High Bias/Low Variance)")
print(f"Training MSE: {mse_train_linear}")
print(f"Testing MSE: {mse_test_linear}\n")
print("Polynomial Regression Model (Low Bias/High Variance)")
print(f"Training MSE: {mse_train_poly}")
print(f"Testing MSE: {mse_test_poly}")

from mlxtend.evaluate import bias_variance_decomp
mse, bias, var = bias_variance_decomp(linear_model, X_train, y_train,
X_test, y_test, loss='mse', num_rounds=200, random_seed=1)
# summarize results
print('\n\n')
print('Bias - Variance of Linear Model')
print('MSE: %.3f' % mse)
print('Bias: %.3f' % bias)
print('Variance: %.3f' % var)

mse, bias, var = bias_variance_decomp(poly_model, X_train, y_train,
X_test, y_test, loss='mse', num_rounds=200, random_seed=1)
# summarize results
print('\n\n')
print('Bias - Variance of Polynomial Model')
print('MSE: %.3f' % mse)
print('Bias: %.3f' % bias)
print('Variance: %.3f' % var)
```

OUTPUT





Linear Model Training MSE: 0.19743335485331392

Linear Model Testing MSE: 0.24695224333039673

Polynomial Model Training MSE: 0.029514806989774718

Polynomial Model Testing MSE: 0.1425297085836628

Linear Regression Model (High Bias/Low Variance)

Training MSE: 0.19743335485331392

Testing MSE: 0.24695224333039673

Polynomial Regression Model (Low Bias/High Variance)

Training MSE: 0.029514806989774718

Testing MSE: 0.1425297085836628

Bias - Variance of Linear Model

MSE: 0.253

Bias: 0.247

Variance: 0.006

Bias - Variance of Polynomial Model

MSE: 2.609

Bias: 0.140

Variance: 2.468

Assignment:

1. In the following segment of code
   poly_model = make_pipeline(PolynomialFeatures(degree=15)
   vary the degree to 1,2,3,11 and comment on bias and variance
   (Only note down MSE, Bias, and Variance for this assignment)
2. In the following segment of code

   ```
   X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
   ```

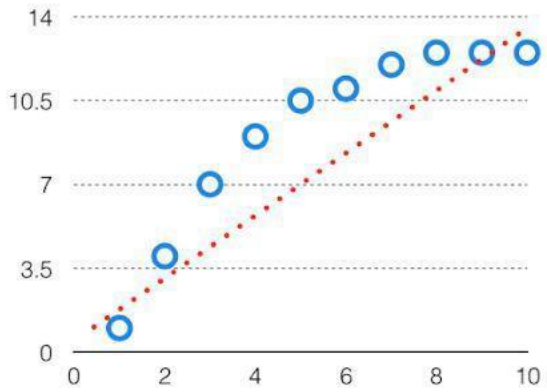
   Very test_size = 0.25,0.3,0.35 and repeat **Assignment question-1** and comment on the results
   (Only note down MSE, Bias, and Variance for this assignment)

Theory:

It is important to understand prediction errors (bias and variance) when it comes to accuracy in any machine-learning algorithm. There is a trade-off between a model's ability to minimize bias and variance which is referred to as the best solution for selecting a value of Regularization constant. A proper understanding of these errors would help to avoid the overfitting and underfitting of a data set while training the algorithm**.**

What is Bias?

The bias is known as the difference between the prediction of the values by the Machine Learning model and the correct value. Being high in biasing gives a large error in training as well as testing data. It recommended that an algorithm should always be low-biased to avoid the problem of underfitting. By high bias, the data predicted is in a straight line format, thus not fitting accurately in the data in the data set. Such fitting is known as the Underfitting of Data. This happens when the hypothesis is too simple or linear in nature. Refer to the graph given below for an example of such a situation.

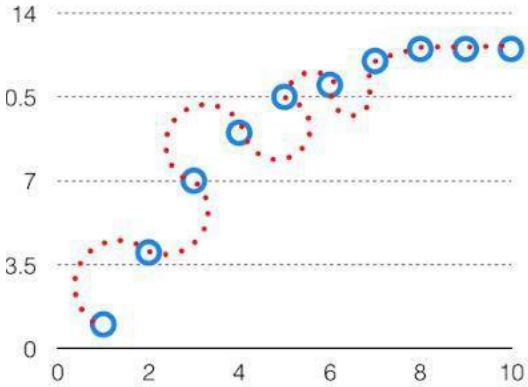


In such a problem, a hypothesis looks like follows.

$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

What is Variance?

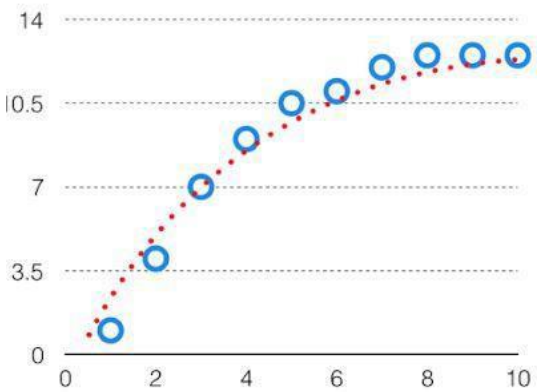
The variability of model prediction for a given data point which tells us the spread of our data is called the variance of the model. The model with high variance has a very complex fit to the training data and thus is not able to fit accurately on the data which it hasn't seen before. As a result, such models perform very well on training data but have high error rates on test data. When a model is high on variance, it is then said to as **Overfitting of Data**. Overfitting is fitting the training set accurately via complex curve and high order hypothesis but is not the solution as the error with unseen data is high. While training a data model variance should be kept low. The high variance data looks as follows.



In such a problem, a hypothesis looks like follows.

Bias Variance Trade-off

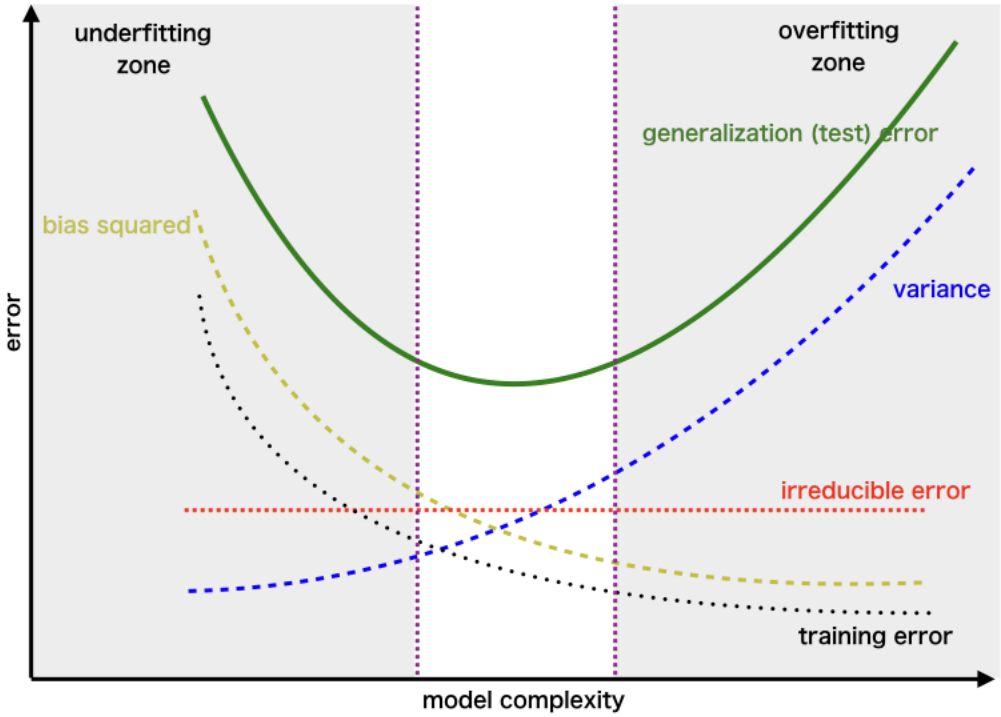
If the algorithm is too simple (hypothesis with linear equation) then it may be on high bias and low variance condition and thus is error-prone. If algorithms fit too complex (hypothesis with high degree equation) then it may be on high variance and low bias. In the latter condition, the new entries will not perform well. Well, there is something between both of these conditions, known as a Trade-off or Bias Variance Trade-off. This trad eoff in complexity is why there is a tradeoff between bias and variance. An algorithm can't be more complex and less complex at the same time. For the graph, the perfect trade off will be like this.



We try to optimize the value of the total error for the model by using the Bias-Variance Trade-off.

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

The best fit will be given by the hypothesis on the trade-off point. The error to complexity graph to show trade-off is given as



Region for the Least Value of Total Error

This is referred to as the best point chosen for the training of the algorithm which gives low error in training as well as testing data.

Expt:7

Aim: **Program to implement a decision tree and display it**

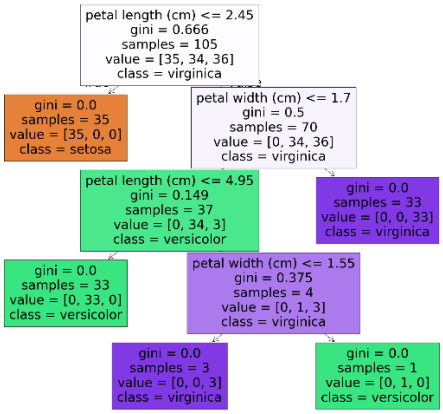
(a) **Program to implement decision tree as a classifier**

```
#PROGRAM To Implement a Decision TREE AND to DISPLAY IT
print('PROGRAM To Implement a Decision Tree AND to DISPLAY IT')
from sklearn import datasets
import pandas as pd
iris=datasets.load_iris()
# df will fold dataset as a table
df=pd.DataFrame(
iris.data,
columns=iris.feature_names
)
#labels are assigned to df[target] table or array
df['target']=pd.Series(
iris.target
)
from sklearn.model_selection import train_test_split
# Train Test Split Ratio
df_train,df_test=train_test_split(df,test_size=0.3)
df['target_names']=df['target'].apply(lambda y:iris.target_names[y])
print('Number of Training samples')
print(df_train.shape[0])
print('Number of Testing samples')
print(df_test.shape[0])
#Importing Decision Tree Classifier
from sklearn.tree import DecisionTreeClassifier
clf=DecisionTreeClassifier()
x_train=df_train[iris.feature_names]
x_test=df_test[iris.feature_names]
y_train=df_train['target']
y_test=df_test['target']
#Training Decision Tree Classifier
clf.fit(x_train,y_train)
#Testing the data
y_test_pred=clf.predict(x_test)
print('Class of Testing Samples')
print(y_test_pred)
#To display the decision tree in command shell
from sklearn.tree import export_text
from sklearn import tree
from matplotlib import pyplot as plt
text_representation = tree.export_text(clf)
print(text_representation)
with open("decistion_tree.log", "w") as fout:
  fout.write(text_representation)
fig = plt.figure(figsize=(25,20))
```

```
_ = tree.plot_tree(clf,
feature_names=iris.feature_names,
class_names=iris.target_names,
filled=True)
fig.savefig("decistion_tree.png")
```

OUTPUT

```
PROGRAM To Implement a Decision Tree AND to DISPLAY IT
Number of Training samples
105
Number of Testing samples
45
Class of Testing Samples
[1 0 2 2 0 2 0 0 0 2 0 2 1 1 2 0 0 2 1 0 1 0 1 1 2 1 2 2 2 0 1 1 1 1 0 2 2
 1 1 0 1 0 2 2 0]
|--- feature_2 <= 2.45
|   |--- class: 0
|--- feature_2 >  2.45
|   |--- feature_3 <= 1.70
|   |   |--- feature_2 <= 4.95
|   |   |   |--- class: 1
|   |   |--- feature_2 >  4.95
|   |   |   |--- feature_3 <= 1.55
|   |   |   |   |--- class: 2
|   |   |   |--- feature_3 >  1.55
|   |   |   |   |--- class: 1
|   |--- feature_3 >  1.70
|   |   |--- class: 2
```



(b) Program to display decision tree and calculate its accuracy

```
#PROGRAM To Calculate Accuracy of Decision Tree
print('PROGRAM To Calculate Accuracy of Decision Tree')
from sklearn import datasets
import pandas as pd
iris=datasets.load_iris()
# df will fold dataset as a table
df=pd.DataFrame( iris.data, columns=iris.feature_names )
#labels are assigned to df[target] table or array
df['target']=pd.Series( iris.target )
from sklearn.model_selection import train_test_split
# Train Test Split Ratio
df_train,df_test=train_test_split(df,test_size=0.3)
df['target_names']=df['target'].apply(lambda y:iris.target_names[y])
print('Number of Training samples')
print(df_train.shape[0])
```

```
print('Number of Testing samples')
print(df_test.shape[0])
#Importing Decision Tree Classifier
from sklearn.tree import DecisionTreeClassifier
clf=DecisionTreeClassifier()
x_train=df_train[iris.feature_names]
x_test=df_test[iris.feature_names]
y_train=df_train['target']
y_test=df_test['target']
#Training Decision Tree Classifier
clf.fit(x_train,y_train)
#Testing the data
y_test_pred=clf.predict(x_test)
print('Class of Testing Samples')
print(y_test_pred)
from sklearn.metrics import accuracy_score
x=accuracy_score(y_test,y_test_pred)
print('Accuracy')
print(x)
```

OUTPUT

PROGRAM To Calculate Accuracy of Decision Tree
Number of Training samples
105
Number of Testing samples
45
Class of Testing Samples
[1 1 1 0 1 1 1 0 2 0 2 0 1 2 2 0 0 2 2 2 1 1 1 1 1 1 1 0 2 1 1 1 1 0 2 2 2
2 2 2 2 2 0 0 2]
Accuracy
0.9333333333333333

(c) **Program to implement decision tree as a regression model**

```
#Program to implement Decision Tree Regression
import numpy as np
n=200
#200 samples
height_pop1_f=np.random.normal(loc=155,scale=10,size=n)
height_pop1_m=np.random.normal(loc=175,scale=5,size=n)
height_pop2_f=np.random.normal(loc=165,scale=10,size=n)
height_pop2_m=np.random.normal(loc=185,scale=5,size=n)
height_f=np.concatenate([height_pop1_f,height_pop2_f])
height_m=np.concatenate([height_pop1_m,height_pop2_m])
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import export_text
```

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
df_height=pd.DataFrame( { 'Gender':[1 for i in range(height_f.size)]+
[2 for i in range(height_m.size)],
'Height':np.concatenate((height_f,height_m))} )
# to calculate mean and median of height
df_height.groupby('Gender')[['Height']].agg([np.mean,np.median]).round(
1)
df_train,df_test=train_test_split(df_height,test_size=0.3)
x_train,x_test=df_train[['Gender']],df_test[['Gender']]
y_train,y_test=df_train['Height'],df_test['Height']
print('Training Samples')
print(df_train)
print('Testing Samples')
print(df_test)
for criterion in['squared_error','absolute_error']:
rgrsr=DecisionTreeRegressor(criterion=criterion)
rgrsr.fit(x_train,y_train)
print(f'criterion={criterion}:\n')
print(export_text(rgrsr,feature_names=['Gender'],spacing=3,decimals=1))
print('Program Executed successfully')
```

OUTPUT

```
Training Samples
   Gender     Height
585      2  183.343569
590      2  171.482107
740      2  188.868396
633      2  182.644465
515      2  178.841086
..     ...       ...
336      1  136.035596
694      2  180.998702
449      2  176.484152
332      1  164.418790
299      1  186.458330

[560 rows x 2 columns]
Testing Samples
   Gender     Height
156      1  146.834942
216      1  156.431498
58       1  155.297595
649      2  188.886167
350      1  151.916364
..     ...       ...
204      1  158.452720
```

```
654      2  180.298237
186      1  145.174562
140      1  174.128847
157      1  134.886183

[240 rows x 2 columns]
criterion=absolute_error:

|--- Gender <= 1.5
|   |--- value: [159.4]
|--- Gender >  1.5
|   |--- value: [179.9]
```

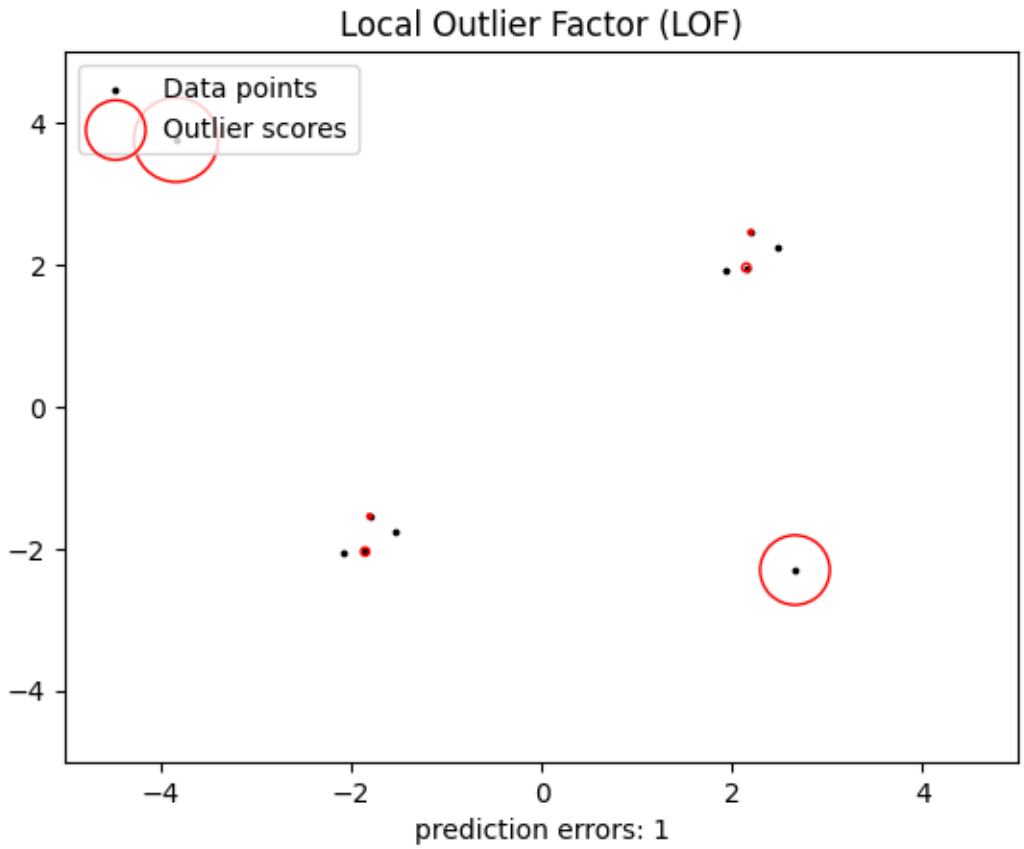
Assignment

1) **Write a program to calculate the confusion matrix and classification report for the decision tree**
2) **Implement a decision tree as a classifier using digits dataset**

```
Expt.8a)Aim:Outlier detection with LOF

import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import LocalOutlierFactor

np.random.seed(42)

# Generate train data
X_inliers = 0.3 * np.random.randn(4, 2) #100 4 -smaples
X_inliers = np.r_[X_inliers + 2, X_inliers - 2]

# Generate some outliers
X_outliers = np.random.uniform(low=-4, high=4, size=(2, 2)) #20
X = np.r_[X_inliers, X_outliers]

n_outliers = len(X_outliers)
ground_truth = np.ones(len(X), dtype=int)
ground_truth[-n_outliers:] = -1

# fit the model for outlier detection (default)
clf = LocalOutlierFactor(n_neighbors=3, contamination=0.1)
# use fit_predict to compute the predicted labels of the training
samples
# (when LOF is used for outlier detection, the estimator has no
predict,
# decision_function and score_samples methods).
y_pred = clf.fit_predict(X)
n_errors = (y_pred != ground_truth).sum()
X_scores = clf.negative_outlier_factor_

plt.title("Local Outlier Factor (LOF)")
plt.scatter(X[:, 0], X[:, 1], color="k", s=3.0, label="Data points")
# plot circles with radius proportional to the outlier scores
radius = (X_scores.max() - X_scores) / (X_scores.max() -
X_scores.min())
plt.scatter(
    X[:, 0],
    X[:, 1],
    s=1000 * radius, #s=1000
    edgecolors="r",
    facecolors="none",
    label="Outlier scores",
)
plt.axis("tight")
plt.xlim((-5, 5))
plt.ylim((-5, 5))
plt.xlabel("prediction errors: %d" % (n_errors))
```

```
legend = plt.legend(loc="upper left")
#legend.legendHandles[0]._sizes = [10]
#legend.legendHandles[1]._sizes = [20]
plt.show()
```

OUTPUT



Assignment:

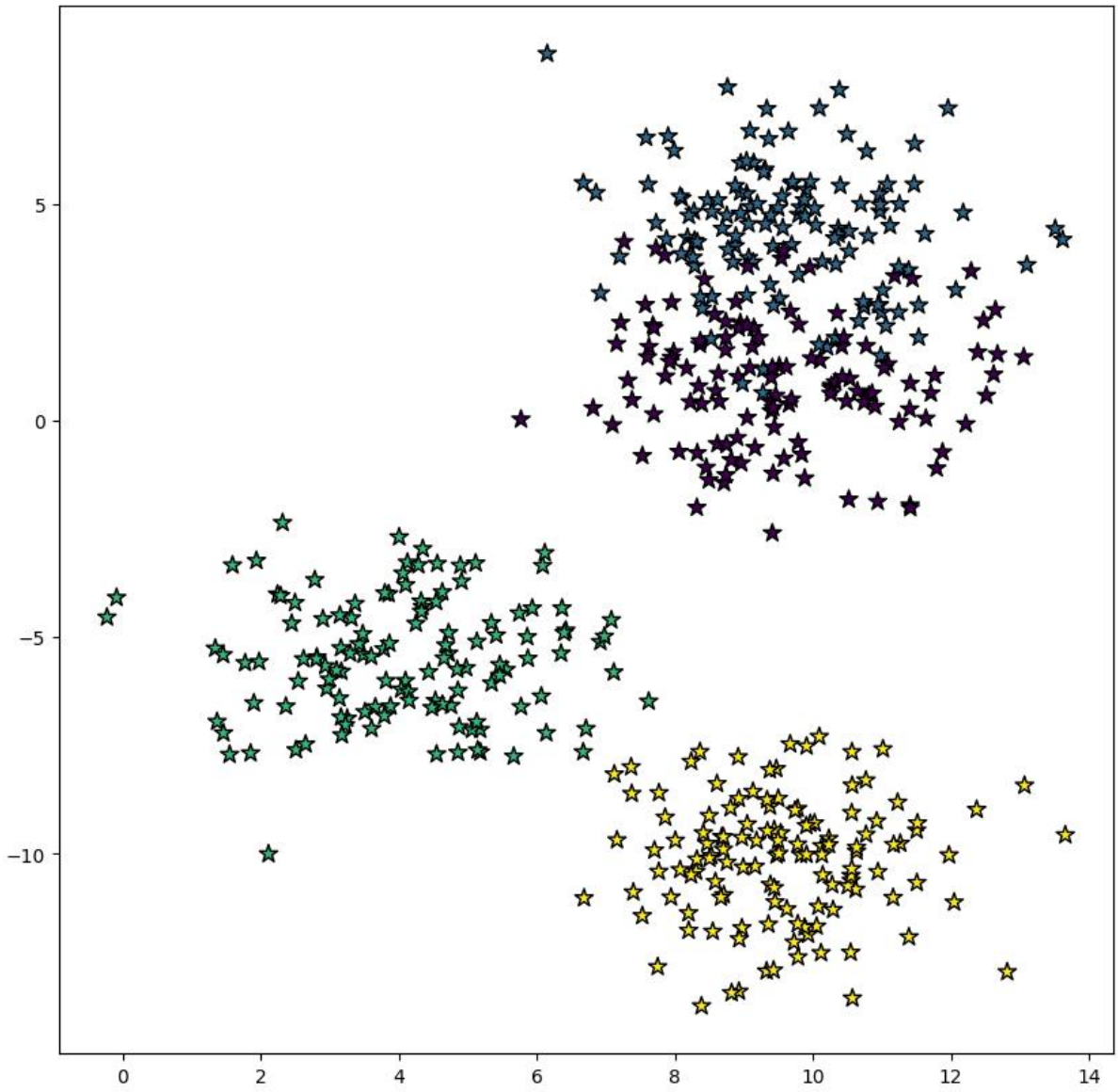
1) **Try changing the number of samples, number of neighbors for finding**
2) **LOF and observe the change in radius.**

Expt.8b) Aim: To design K-Nearest Neighbour Classifier.

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.datasets import make_blobs
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
```

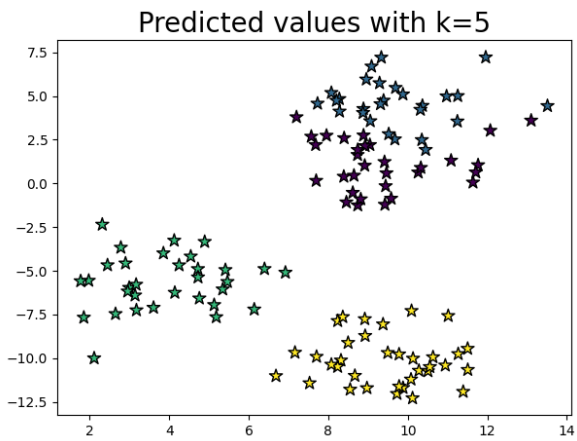
```
X, y = make_blobs(n_samples = 500, n_features = 2, centers =
4,cluster_std = 1.5, random_state = 4)

#plt.style.use('seaborn')
plt.figure(figsize = (10,10))
plt.scatter(X[:,0], X[:,1], c=y, marker= '*',s=100,edgecolors='black')
plt.show()

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state
= 0)

knn5 = KNeighborsClassifier(n_neighbors = 5)
knn1 = KNeighborsClassifier(n_neighbors=1)

knn5.fit(X_train, y_train)
knn1.fit(X_train, y_train)

y_pred_5 = knn5.predict(X_test)
y_pred_1 = knn1.predict(X_test)

from sklearn.metrics import accuracy_score
print("Accuracy with k=5", accuracy_score(y_test, y_pred_5)*100)
print("Accuracy with k=1", accuracy_score(y_test, y_pred_1)*100)

plt.figure(figsize = (15,5))
plt.subplot(1,2,1)
plt.scatter(X_test[:,0], X_test[:,1], c=y_pred_5, marker= '*',
s=100,edgecolors='black')
plt.title("Predicted values with k=5", fontsize=20)

plt.subplot(1,2,2)
plt.scatter(X_test[:,0], X_test[:,1], c=y_pred_1, marker= '*',
s=100,edgecolors='black')
plt.title("Predicted values with k=1", fontsize=20)
plt.show()
```

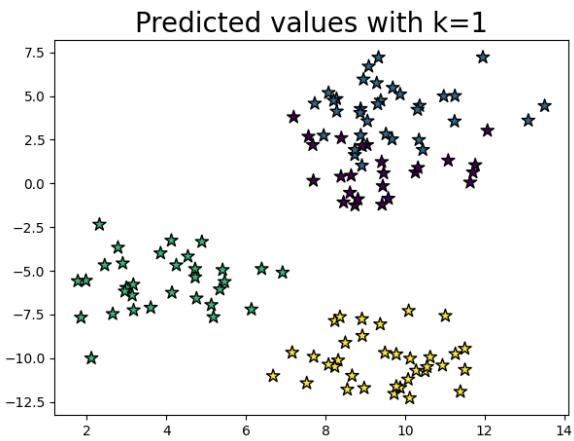
OUTPUT



Accuracy with k=5 93.60000000000001

Accuracy with k=1 90.4





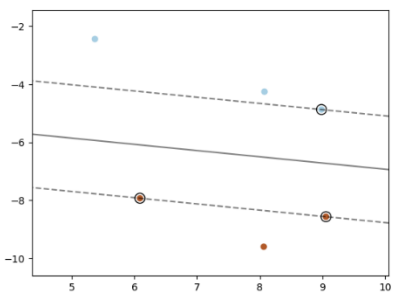
Assignment:

1) **Try KNN algorithm for K=1,3,5,9,11,13 and compare the accuracy**
2) **Change the number of samples and repeat Q.1**

9a) Aim: Implementation of linear SVM

```
import matplotlib.pyplot as plt
from sklearn import svm
from sklearn.datasets import make_blobs
from sklearn.inspection import DecisionBoundaryDisplay
# we create 6 separable points
X, y = make_blobs(n_samples=6, centers=2, random_state=6) #X, y =
make_blobs(n_samples=40, centers=2, random_state=6)
# fit the model, dont regularize for illustration purposes
clf = svm.SVC(kernel= "linear", C=9000) #C=1000
clf.fit(X, y)
plt.scatter(X[:, 0], X[:, 1], c=y, s=30, cmap=plt.cm.Paired)
# plot the decision function
ax = plt.gca()
DecisionBoundaryDisplay.from_estimator(
clf,
X,
plot_method="contour",
colors="k",
levels=[-1, 0, 1],
alpha=0.5,
linestyles=["--", "-", "--"],
ax=ax,
)
# plot support vectors
ax.scatter(
clf.support_vectors_[:, 0],
clf.support_vectors_[:, 1],
s=100,
linewidth=1,
facecolors="none",
edgecolors="k",
)
plt.show()
```

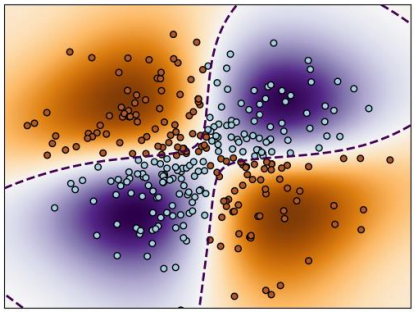
OUTPUT



9b) Aim: Implementation of nonlinear SVM

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm
xx, yy = np.meshgrid(np.linspace(-3, 3, 500), np.linspace(-3, 3, 500))
np.random.seed(0)
X = np.random.randn(300, 2)
Y = np.logical_xor(X[:, 0] > 0, X[:, 1] > 0)
# fit the model
clf = svm.NuSVC(gamma="auto")
clf.fit(X, Y)
# plot the decision function for each datapoint on the grid
Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
plt.imshow(
Z,
interpolation="nearest",
extent=(xx.min(), xx.max(), yy.min(), yy.max()),
aspect="auto",
origin="lower",
cmap=plt.cm.PuOr_r,)
contours = plt.contour(xx, yy, Z, levels=[0], linewidths=2,
linestyles="dashed")
plt.scatter(X[:, 0], X[:, 1], s=30, c=Y, cmap=plt.cm.Paired,
edgecolors="k")
plt.xticks(())
plt.yticks(())
plt.axis([-3, 3, -3, 3])
#Perform binary classification using non-linear SVC with RBF kernel.
The target to predict is a XOR of the inputs.
import numpy as np
plt.show()
```

OUTPUT



Assignment:

1) **Experiment for Number of samples N= 20,100,500 and give inference on the results obtained.**
2) **For N=20 , apply different kernels and C-Values and give your inference on the results obtained.**

ML Lab - Experiment 10

10a: kmeans clustering

```
import matplotlib.pyplot as plt

x = [4, 5, 10, 4, 3, 11, 14 , 6, 10, 12]
y = [21, 19, 24, 17, 16, 25, 24, 22, 21, 21]

plt.scatter(x, y)
plt.show()


from sklearn.cluster import KMeans

data = list(zip(x, y))
inertias = []

for i in range(1,11):
    kmeans = KMeans(n_clusters=i)
    kmeans.fit(data)
    inertias.append(kmeans.inertia_)

plt.plot(range(1,11), inertias, marker='o')
plt.title('Elbow method')
plt.xlabel('Number of clusters')
plt.ylabel('Inertia')
plt.show()



kmeans = KMeans(n_clusters=2)
kmeans.fit(data)

plt.scatter(x, y, c=kmeans.labels_)
plt.show()
```

10.b. Principal Component Analysis(PCA):

```
# importing required libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd


# importing or loading the dataset
dataset = pd.read_csv('Wine.csv')

# distributing the dataset into two components X and Y
X = dataset.iloc[:, 0:13].values
y = dataset.iloc[:, 13].values


# Splitting the X and Y into the
# Training set and Testing set
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 0)


# performing preprocessing part-fitting standard scale
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()

X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
# Applying PCA function on training
# and testing set of X component
from sklearn.decomposition import PCA

pca = PCA(n_components = 2)

X_train = pca.fit_transform(X_train)
X_test = pca.transform(X_test)

explained_variance = pca.explained_variance_ratio_


# Fitting Logistic Regression To the training set
from sklearn.linear_model import LogisticRegression

classifier = LogisticRegression(random_state = 0)
classifier.fit(X_train, y_train)
```

```
▾ LogisticRegression
LogisticRegression(random_state=0)
```

```
# Predicting the test set result using
# predict function under LogisticRegression
y_pred = classifier.predict(X_test)


# making confusion matrix between
#  test set of Y and predicted value.
from sklearn.metrics import confusion_matrix
```

Assignment for 10(a)

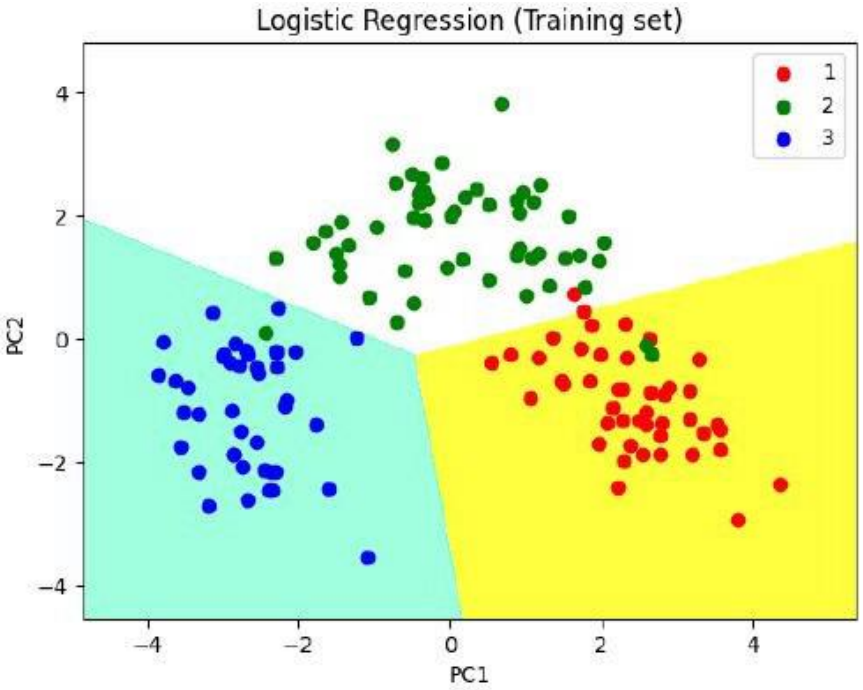
1. Modify no. of samples and k value and give your inference

Assignment for 10(b)

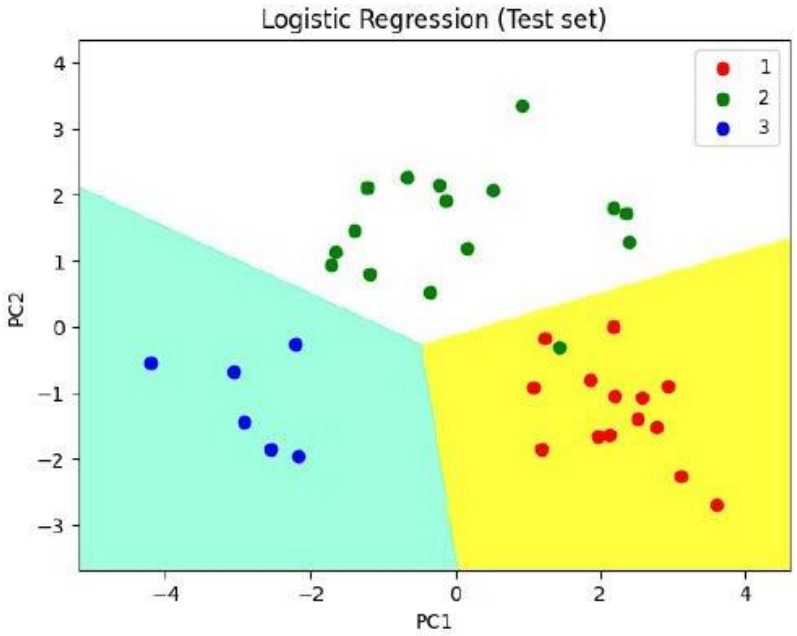
1. **Try for different Train Test split and different data set and give your inference**

```
cm = confusion_matrix(y_test, y_pred)


# Predicting the training set
# result through scatter plot
from matplotlib.colors import ListedColormap

X_set, y_set = X_train, y_train
X1, X2 = np.meshgrid(np.arange(start = X_set[:, 0].min() - 1,
                     stop = X_set[:, 0].max() + 1, step = 0.01),
                     np.arange(start = X_set[:, 1].min() - 1,
                     stop = X_set[:, 1].max() + 1, step = 0.01))

plt.contourf(X1, X2, classifier.predict(np.array([X1.ravel(),
             X2.ravel()]).T).reshape(X1.shape), alpha = 0.75,
             cmap = ListedColormap(('yellow', 'white', 'aquamarine')))



plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())


for i, j in enumerate(np.unique(y_set)):
   plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1],
             c = ListedColormap(('red', 'green', 'blue'))(i), label = j)


plt.title('Logistic Regression (Training set)')
plt.xlabel('PC1') # for Xlabel
plt.ylabel('PC2') # for Ylabel
plt.legend() # to show legend


# show scatter plot
plt.show()
```

```
<ipython-input-33-db6bef8bffd0>:19: UserWarning: *c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided
  plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1],
```




```
# Visualising the Test set results through scatter plot
from matplotlib.colors import ListedColormap

X_set, y_set = X_test, y_test

X1, X2 = np.meshgrid(np.arange(start = X_set[:, 0].min() - 1,
                     stop = X_set[:, 0].max() + 1, step = 0.01),
                     np.arange(start = X_set[:, 1].min() - 1,
                     stop = X_set[:, 1].max() + 1, step = 0.01))

plt.contourf(X1, X2, classifier.predict(np.array([X1.ravel(),
             X2.ravel()]).T).reshape(X1.shape), alpha = 0.75,
             cmap = ListedColormap(('yellow', 'white', 'aquamarine')))

plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())

for i, j in enumerate(np.unique(y_set)):
    plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1],
                c = ListedColormap(('red', 'green', 'blue'))(i), label = j)

# title for scatter plot
plt.title('Logistic Regression (Test set)')
plt.xlabel('PC1') # for Xlabel
plt.ylabel('PC2') # for Ylabel
plt.legend()

# show scatter plot
plt.show()
```



Ref:

1. https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html
2. https://scikit-learn.org/stable/modules/clustering.html
3. https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html
4. https://www.kdnuggets.com/2023/05/principal-component-analysis-pca-scikitlearn.html
5. https://builtin.com/machine-learning/pca-in-python